

4

MLP Representational Capabilities

The standard multilayer perceptron (MLP) is a cascade of single-layer perceptrons (figure 4.1). There is a layer of input nodes, a layer of output nodes, and one or more intermediate layers. The interior layers are sometimes called "hidden layers" because they are not directly observable from the system inputs and outputs. Each node has a response $f(\mathbf{w}^T \mathbf{x})$ where $\mathbf{x}$ is the vector of output activations from the preceding layer, $\mathbf{w}$ is a vector of weights, and f is a bounded nondecreasing nonlinear function such as the sigmoid. Normally, one of the weights acts as a bias by virtue of connection to a constant input. Nodes in each layer are fully connected to nodes in the preceding and following layers. There are no connections between units in the same layer, connections from one layer back to a previous layer, or "shortcut" connections that skip over intermediate layers. Although back-propagation can be applied to more general networks, this is the most commonly used structure.

The following sections summarize some properties and limitations that result from this structure, independent of methods used to set the weights.

How to Count Layers?

A minor digression: there is some disagreement about how to count layers in a network. Some say a network with one hidden layer is a three-layer network because there are three layers of nodes: the inputs, the hidden units, and the outputs. Others say this is a two-layer network because there are only two layers of active nodes, the hidden units and outputs. Inputs are excluded because they do no computation. We tend to follow this convention and say that an L -layer network has L active layers; that is, $L - 1$ hidden layers and an output layer. Conveniently, this is also the number of weight layers. Not everyone uses the same convention, however, so it is often simplest to explicitly specify the number of hidden layers. The network in figure 4.1, for example, would be called a two-hidden-layer network. In spite of the convention, it is natural to refer to the input layer at times; we did so in the first paragraph of this chapter.

The notation $N_1/N_2/\dots/N_L$ is sometimes used to describe the structure of a layered network. This is simply a list of the number of nodes in each layer. A 10/3/2 network, for example, has 10 inputs, 3 nodes in a hidden layer, and 2 outputs. A 16/10/5/1 network would have 16 inputs, 10 nodes in the first hidden layer, 5 nodes in the second hidden layer, and 1 output. Unless otherwise specified, each layer is presumed to be fully connected to the preceding and following layers with no short-cut or feedback connections. Figure 4.1 illustrates a 5/5/3/4 structure.

4.1 Representational Capability

The representational capability of a network can be defined as the range of mappings it can implement when the weights are varied. A particular mapping is within the representational

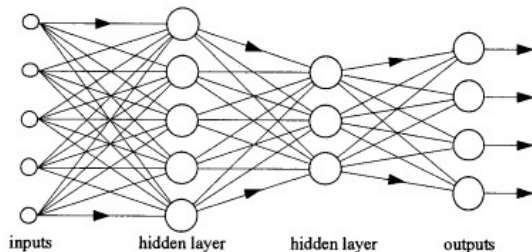


Figure 4.1

MLP structure. A multilayer perceptron is a cascade of single-layer perceptrons. There is a layer of input nodes and a layer of output nodes with one or more hidden layers in between.

capability of a net if there is a set of weights for which the net performs the mapping. Theories about representational capability answer questions like: Given a set of inputs and associated target outputs, is there a set of weights that allow the network to perform the desired mapping? Is the network capable of generating the desired outputs from the given inputs? In some cases, an exact solution may be required while in others, an approximation may be allowed. Results say whether a particular problem might be solvable by a particular network without necessarily producing the weights that generate the solution. If the answer is negative, we know the network cannot solve the problem and this saves us the expense of a futile search. If the answer is positive, we know a solution exists but these results don't guarantee that it will be easy to find or that a chosen optimization method will be able to find it. Results of this kind provide broad guidelines in selecting an architecture for a problem. One well-known representational result, for example, is that single-layer networks are capable of representing only linearly separable functions.

Two Hidden Layers Are Sufficient

When designing a layered network, an obvious first question is how many layers to use. Lippmann [247] shows that two hidden layers are sufficient to create classification regions of any desired shape. In figure 4.2, linear threshold units in the first hidden layer divide the input space into half-spaces with hyperplanes, units in the second hidden layer AND (form intersections of) these half-spaces to produce convex regions, and the output units OR (form unions of) the convex regions into arbitrary, possibly unconnected, shapes. Given a sufficient number of units, a network can be formed that divides the input space in any way desired, producing 0 when the input is in one region and 1 when it is in the other. The boundaries are piecewise linear, but any smooth boundary can be approximated with enough units.

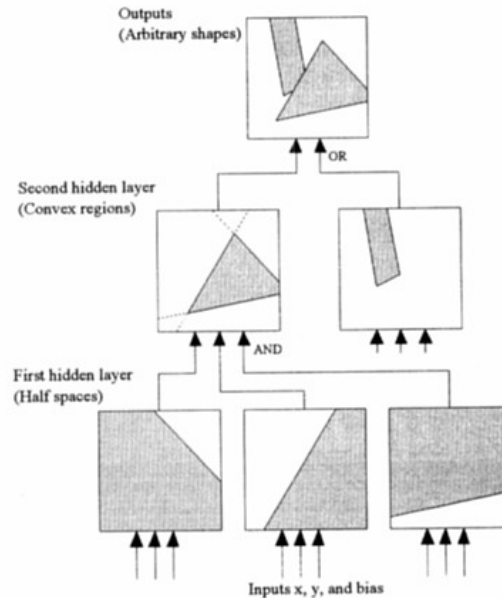


Figure 4.2

Three layers of linear threshold units are sufficient to define arbitrary regions. Units in the first hidden layer divide the input space with hyperplanes, units in the second hidden layer can form convex regions bounded by these hyperplanes, and output units can combine the regions defined by the second layer into arbitrarily shaped, possibly unconnected, regions. Here, the boundaries are piecewise linear, but any smooth boundary can be approximated with enough units (based on [247].)

To approximate continuous functions, one can add and subtract (rather than logically OR) convex regions with appropriate weighting factors so two hidden layers are also sufficient to approximate any desired bounded continuous function [234].

One-hidden-layer Nets Can Represent Nonconvex, Disjoint Regions

Units in the second hidden layer of figure 4.2 respond to convex regions in the input space. Because these would be the outputs in a single-hidden-layer network, it is sometimes mistakenly assumed that single-hidden-layer networks can recognize only convex decision regions.

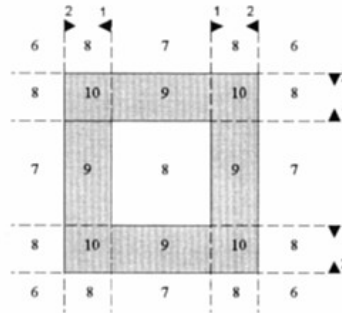


Figure 4.3

Single-hidden-layer networks can recognize nonconvex decision regions. Here, eight hidden units in a single layer create a nonconvex "square donut." Each dashed line is the decision boundary of one of the hidden units. The hidden unit is active (1) on the side of the line indicated by the pointer and connects to the output unit with the weight indicated next to the pointer. The other numbers show the summed input to the output unit in different regions. Thresholding at 8.5 creates the square donut.

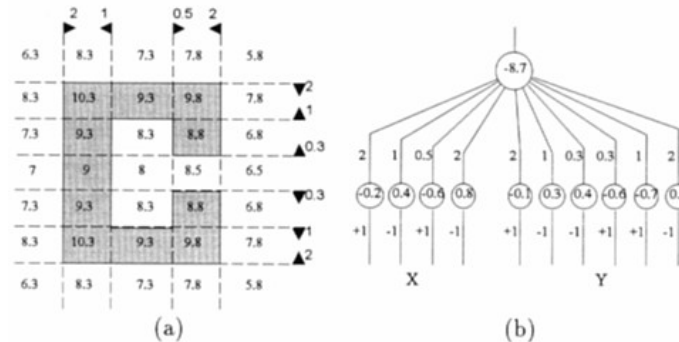


Figure 4.4

Weiland and Leighton [407] provide this example of a nonconvex region recognized by a single-hidden-layer network. Thresholding at 8.7 creates the 'C' shaped region: (a) a nonconvex decision region, and (b) the network. There are 10 hidden nodes. Each has one connection with weight $w = \pm 1$ from either the x or y input. Numbers inside the circles are the node thresholds.

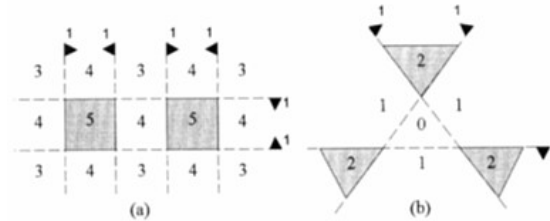


Figure 4.5

Disjoint regions recognized by single-hidden-layer networks.

In addition to nonconvex regions, single-hidden-layer networks can also recognize disjoint (unconnected) regions.

Counter-examples are shown in figures 4.3 through 4.5. Figures 4.3 and 4.4 illustrate nonconvex regions recognized by three-layer nets. (Figure 4.4 is derived from Weiland and Leighton [407].) Figure 4.5 shows examples of disjoint (unconnected) regions recognizable by three-layer networks. Other examples can be found in [180, 255, 249].

4.2 Universal Approximation Capabilities

The examples above suggest that any bounded function can be approximated with arbitrary accuracy if enough hidden units are available. These ideas have been extended theoretically to show that multilayer perceptrons are *universal approximators*. That is, they are capable of arbitrarily accurate approximation of essentially arbitrary continuous mappings from the $[-1, +1]^n$ hypercube to the $(-1, 1)$ interval. This is an important result because it says neural networks, as a class, are powerful enough to implement essentially any function we require.

4.2.1 Kolmogorov's Theorem, One Hidden Layer Is Sufficient

A somewhat surprising result is that two hidden layers are not necessary for universal approximation; one hidden layer is sufficient. Kolmogorov [219, 220] showed that a continuous function of several variables can be represented *exactly* by the superposition of continuous one-dimensional functions of the original input variables. A refined proof by Sprecher [355] is often cited. Hecht-Nielsen [161] introduced the result to the neural network community and showed an implementation in the form of a single-hidden-layer network. Briefly, the result is that any continuous function mapping an n -dimensional input, $n \geq 2$, to an m -dimensional output can be implemented exactly by a network with one

hidden layer. For $\phi: I^n \rightarrow \mathbb{R}^m$, $\phi(\mathbf{x}) = \mathbf{y}$, where I is the closed interval $[0, 1]$ and therefore I^n is an n -dimensional hypercube, the function ϕ can be implemented *exactly* by a single-hidden-layer neural network having n elements in the input layer, $(2n + 1)$ elements in the middle layer, and m elements in the output layer. That is,

$$z_k = \sum_{j=1}^n \lambda_j^k \psi(x_j + \epsilon k) + k \quad (4.1)$$

$$y_i = \sum_{k=1}^{2n+1} g_i(z_k), \quad (4.2)$$

where ϵ is a real constant, ψ is a continuous real monotonic increasing function independent of ϕ but dependent on n , and λ_j^k is a rational number, $0 < \epsilon \leq \delta$, where δ is an arbitrarily chosen positive constant. The output node functions g_i are real and continuous and depend on ϕ and δ . Unfortunately, the proof is not constructive; it does not say how weights should be chosen to implement a particular mapping or specify the functions g_i (different for each i), which have unknown and often extremely nonlinear shapes.

Girosi and Poggio [139] point out that the theorem might not be relevant because the inner functions ψ are extremely nonsmooth. (The theorem fails when restricted to smooth functions.) The functions are very unlike the sigmoids normally used in neural networks and not likely to be learnable because of their extreme roughness. Further, the functions g_i depend on ϕ , are all different, and do not form a parameterized family. They are likely to be at least as complex as ϕ , if not more so, and therefore no easier to learn than ϕ itself. In effect, the complexity of approximating the original function is shifted into the task of finding the internal node functions.

In more recent work, Sprecher [356] describes a stronger version of the theorem using translates of a single function in place of the multiple internal functions g_k . The replacement function remains nonsmooth, however, so the objections remain.

Kurkova [231] notes that although exact representation is not possible with smooth internal functions, approximation is, and goes on to show universal approximation properties of sigmoidal networks with two hidden layers. But other work [244] suggests that approximation of the internal functions does not yield approximation of the target function.

4.2.2 Other Proofs of Universal Approximation Capability

Several proofs are based on showing that single-hidden-layer networks can implement Fourier transforms and thus have the same approximation capabilities. The idea is that a sinusoid can be implemented by sums and differences of many shifted sigmoids, one for each half cycle. Irie and Miyake [186] give a proof based on Fourier integrals where the

number of nodes required for exact approximation is infinite. Gallant and White [132] give another Fourier transform proof using monotone "cosine-squasher" sigmoids. Funahashi [131] approximates the integral representation of Irie and Miyake with a discrete sum and shows that networks with at least one hidden layer of sigmoid units are universal approximators.

Cybenko [95, 96] provided one of the first rigorous proofs of universal approximation capability, showing that a network with linear output units and a single hidden layer of sigmoid units is sufficient for uniform approximation of any continuous function in the unit hypercube. Hornik, Stinchcombe, and White [177] and Stinchcombe and White [358] show that standard multilayer networks using arbitrary squashing functions are universal approximators; single-hidden-layer nets are included as a special case. Approximation of a function and its derivatives using arbitrary bounded nonconstant squashing functions is discussed by Hornik [175]. Other work on approximation includes [162, 48, 188, 226, 176].

In most cases, these are existence proofs rather than constructive recipes for finding a solution to a particular problem. These results do not guarantee that a chosen optimization method will find the necessary weights or that the solution found will be efficient. A constructive proof based on the Radon transform is given by Carroll and Dickson [64].

It should be noted that universal approximation is not a rare property. Many other systems have similar capabilities: polynomials, trigonometric polynomials (e.g., Fourier series), kernel regression systems, wavelets, and so on. By itself, this property does not make neural networks special. The results are important because they show that neural networks are powerful enough to approximate most functions that people find interesting. The lack of a universal approximation capability, on the other hand, would be bad news; neural networks would then be too weak for many problems and therefore much less appealing.

Given that there are many universal approximation systems, the choice of one over another depends on other factors such as efficiency, robustness, and so on. Barron [21, 22, 23] addresses the problem of how the MLP approximation error scales with the number of training samples and the number of parameters. One important result is that the error decreases like $N^{-1} O(Mp/\epsilon)$. Here N is the number of samples, M is the number of hidden nodes, p is the input dimension (so M_p is approximately the number of parameters), and ϵ is the desired approximation error.

One Hidden Layer Is Not Always Enough The limits of these proofs are sometimes forgotten. They say that a sufficiently large one-hidden-layer network is capable of essentially arbitrarily accurate approximation of continuous functions over compact regions. Although this covers most functions we would like to approximate, it may not include them all. Sontag [350, 351] points out that there are certain functions, important in inverse control, which cannot be approximated by single-hidden-layer nets with any number of units but which can be implemented rather simply with two hidden layers. That is, there are control systems that can be stabilized by a two-hidden-layer network but not by a one-hidden-layer net. The difference depends not on the number of units needed to achieve a certain numerical accuracy but rather on the need to approximate discontinuous functions that may arise as one-sided inverses of continuous functions.

4.3 Size versus Depth

Since a single sufficiently large hidden layer is adequate for approximation of most functions, why would anyone ever use more? One reason hangs on the words "sufficiently large." Although a single hidden layer is optimal for some functions, there are others for which a single-hidden-layer solution is very inefficient compared to solutions with more layers.

Certain functions containing disjoint decision regions cannot be realized exactly with only a single hidden layer of threshold units [255]. Certain functions can be implemented exactly by small networks with two hidden layers but require an infinite number of nodes to approximate with a single hidden layer network [255, 254]. Figure 4.6 shows an example from [255]. Gibson and Cowan [137] provide another example. Chester [80] describes a "pinnacle" function (1 at the origin and 0 elsewhere) for which a single-hidden-layer network needs $O(1/\epsilon)$ nodes to achieve $O(\epsilon)$ maximum error where a two-hidden-layer network needs only 4 nodes. This uses a maximum error criteria, however, rather than mean squared error. For certain functions, single-hidden-layer networks may require very large weights or extreme precision [180].

Although these results suggest that two-hidden-layer networks are more powerful, one-hidden-layer networks may be adequate for many functions encountered in practice. One-and two-hidden-layer networks are compared empirically by de Villiers and Barnard [104]. For fair comparison, the networks are configured to have approximately the same number of weights. Using conjugate gradient training and artificial clustered 2-dimensional data (mixtures of Gaussians), no significant difference was found in the best solution found by either architecture. The one-hidden-layer nets were reported to have lower average error and perhaps generalized better. For two-hidden-layer nets, the same authors report that

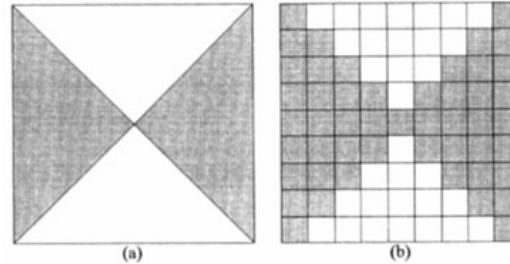


Figure 4.6

Certain functions can be implemented exactly by small networks with two hidden layers but require an infinitenumber of nodes to approximate if constrained to a single hidden layer [255, 254]: (a) a simple function that canbe implemented with two hidden layers each containing two threshold units, (b) an approximation by a networkwith just one hidden layer, (from [255]).

training is easier if the hidden layers have approximately equal sizes. Similar results are reported by Huang and Lippmann [180]; no significant difference was seen in error rate or convergence time.

4.3.1 Size versus Depth for Boolean Functions

Many results exist for Boolean logic functions, e.g., [347]. (See figure 4.7 for the elementary gates.) It is obvious that a net with one hidden layer of 2^n threshold units can compute any Boolean function of n binary inputs since any Boolean function can be expressed in conjunctive (or disjunctive) normal form. Each hidden unit computes one of the 2^n product terms and the output unit ORs selected product terms to produce the desired result. An AND-OR array has this structure. This amounts to exhaustive enumeration of all positive cases and becomes impractical for large n .

More economical solutions can often be obtained by adding more layers. The number of nodes needed can be reduced by a factor of $O(2^{n/2}/\sqrt{n})$ [347: 125]. Notice that this is an exponential reduction in size.

Because a one-hidden-layer solution may be inefficient, there may be functions that cannot be implemented by a network of limited size using just one hidden layer. It is known that there are Boolean functions which cannot be computed by threshold networks with

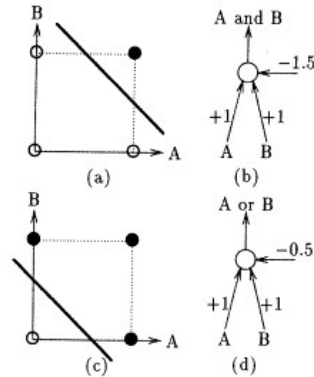


Figure 4.7

AND and OR logic functions. Any logic function can be implemented by a circuit of AND, OR, and NOT gates. Threshold units are at least as powerful as logic circuits because they can implement all the necessary elementary functions, as well as others: (a) the AND function, (b) linear threshold unit implementation of AND, (c) the OR function, and (d) linear threshold unit implementation of OR. The NOT function can be obtained from a single-input unit with a large negative weight and a zero threshold.

one hidden layer and a constant number of nodes, but which can be computed by threshold networks with two hidden layers and a constant number of nodes ([250]; cited in [99]).

"Symmetric" functions depend only on the sum of the inputs, $\sum_i x_i$; parity is one example. For arbitrary symmetric Boolean functions of n inputs, $O(n)$ units are needed in depth-2 (one hidden layer) networks of linear threshold units (LTU), but only $O(\sqrt{n/\log n})$ LTU gates are needed to compute general symmetric functions if there is no restriction on depth [347].

Another reason to use more than one hidden layer is to decrease the weight magnitudes. A one-hidden-layer implementation may require arbitrarily large weights. If permissible weight values are bounded (e.g., by hardware limitations), networks with more hidden layers may be more efficient. A depth- d circuit with exponential weights can be simulated by a depth- $(d+1)$ circuit with polynomially bounded weights at the cost of a polynomial increase in network size [347: 40]. A single linear threshold unit with exponential weights

can be exchanged for a depth-3 polynomial size circuit with polynomial weights [347: 41]. Anything computable by a depth-2 threshold circuit of polynomial size can be computed by depth-3 *small weight* threshold circuits of polynomial size ([143]; cited in [142]). Thus if the range of weights is limited, networks with more than one hidden layer may be able to realize functions that cannot be realized by single-hidden-layer networks.

Caveats

The results just stated say there are functions for which single-hidden-layer networks are less efficient than networks with more hidden layers. Of course, there are still functions where small single-hidden-layer networks are optimal and additional hidden layers are not useful. Single-hidden-layer networks may need large numbers of nodes to compute arbitrary functions, but small networks may suffice for particular functions.

Many of these results are statements about the power of a class of networks rather than guarantees that a particular network will be able to learn a particular set of data and generalize accurately. Many are based on asymptotic analyses valid only for large data sets or large input dimensions. Many are statements about the existence of a solution rather than constructive statements about how to find the necessary set of weights. Most depend on particular assumptions that may be violated in a given problem.

In real problems, training sets may be rather small and nonrandomly sampled, data distributions may have arbitrary forms, and the target function is unknown. It may be problematic just to determine if the data fits the assumptions. Although representational capability results can be helpful in putting bounds on the size and configuration of a network, they do not guarantee that a particular network and training algorithm will be able to learn a particular set of samples of a given function. Even if it is guaranteed that a particular network can exactly classify N training points (using some set of weights), this does not necessarily imply that the system will generalize well to new points. Finally, many of the results for Boolean functions are for exact implementation and may not hold when approximation is allowed.

4.4 Capacity versus Size

Another big question in designing a network is how many nodes to place in each layer. Universal approximation results say that a sufficiently large network can fit almost any function we are likely to want to approximate, but they do not deal with the problem of training a network to fit a finite set of discrete data points. Obviously, we cannot have infinite numbers of nodes in practice so it is useful to have bounds on the number that will be needed to fit a particular function. Even if arbitrarily large networks were allowed, it might not be possible to use them effectively given the limited amount of information contained in the training samples. After the network grows past a certain size,